AUTOSAR High-Level Design (HLD) Document

Project: ADAS Control System | Version 2.0

NOTICE: Synthetic Demo HLD — For Demonstration Only. Not an official AUTOSAR specification.

Document Identifier: HLD-ADAS-2026-X89
Classification: Confidential / Engineering Baseline
Author: Systems Architecture Engineering Team
ECU Target: ADAS Domain Controller (TriCore AURIX / ARM Cortex-R52)

1.0 Document Revision History

Version	Date	Author	Description
1.0	2026-01-15	Arch Team	Initial baseline release of ADAS controller architecture.
2.0	2026-06-20	Lead Architect	Upgraded sensor fusion pipeline to SensorFusionV2. Updated brake interface contracts and protocols.

2.0 System Architecture Overview

The ADAS Control System provides Level 2+ automated driving capabilities including Autonomous Emergency Braking (AEB), Adaptive Cruise Control (ACC), and Lane Centering Assist (LCA). The architectural model follows AUTOSAR Classic and Adaptive methodologies with strictly delineated software component (SWC) boundaries, standardized RTE communication ports, and ISO 26262 ASIL-D safety integrity.

The system executes across a multi-core automotive microcontroller. Real-time sensor perception data is ingested from external radar and optical front-camera sensors, synchronized, and processed to calculate safety trajectories and deceleration commands.

Primary Architecture Goals:

• Deterministic execution with maximum 20ms end-to-end latency from optical perception to brake actuation.
• Full fault isolation between perception layers and actuator execution.
• Compliance with automotive bus protocols (CAN-FD and Automotive Ethernet 100BASE-T1).

3.0 Software Component (SWC) Specifications

The application layer decomposes functionality into modular AUTOSAR software components. Each component exposes standardized Sender-Receiver and Client-Server ports configured via the Runtime Environment (RTE).

3.1 Component: CameraManager

Software Component: CameraManager is responsible for deserializing raw video streams from the front-facing monocular camera module. It performs object boundary detection, optical flow estimation, and lane boundary classification. It provides the interface CameraData and transmits high-bandwidth visual object frames over Automotive Ethernet.

3.2 Component: RadarProcessor

Software Component: RadarProcessor connects to long-range and medium-range radar transceivers. It filters raw radar reflections, performs Doppler velocity resolution, and clusters target point clouds. It provides the interface RadarData to downstream fusion SWCs.

3.3 Component: SensorFusion

Software Component: SensorFusionV2 performs spatial and temporal alignment of optical objects from CameraManager and radar clusters from RadarProcessor. Using an Extended Kalman Filter (EKF) and Hungarian association algorithm, SensorFusionV2 synthesizes a unified environmental model. It consumes CameraData from CameraManager and RadarData from RadarProcessor. It outputs the interface FusedObjectData.

Architectural Reference: For functional safety diagnostics, SensorFusionV2 periodically outputs interface ObjectFusionStatus to monitor Kalman covariance convergence. (Note: Port configuration pending review).

Data Flow Description:

CameraManager -> SensorFusion via Ethernet
RadarProcessor -> SensorFusion via CAN-FD
SensorFusion -> ADASController via Autosar RTE

3.4 Component: ADASController

Software Component: ADASController implements high-level supervisory decision logic, situation assessment, and motion planning. It evaluates target vehicle trajectories against host vehicle dynamics and assesses time-to-collision (TTC). ADASController consumes FusedObjectData from SensorFusion and VehicleState from VehicleInterface.

When collision hazards exceed safety thresholds, ADASController generates autonomous deceleration and torque commands. It transmits commands to the braking subsystem.

Documented Architectural Dependencies:

• ADASController depends on SensorFusion
• ADASController depends on BrakeController
• ADASController depends on VehicleInterface

3.5 Component: BrakeController

Software Component: BrakeController operates on the chassis safety partition. It coordinates electronic stability control (ESC) and electro-hydraulic braking actuators to achieve requested vehicle deceleration.

BrakeController consumes the interface BrakeControlCommand from ADASController over Chassis CAN. Signal payload specifies target deceleration rate in m/s^2 and prefill pressure targets.

3.6 Component: VehicleInterface

Software Component: VehicleInterface connects to the central vehicle CAN gateway. It samples vehicle dynamics sensors including wheel speed sensors, inertial measurement unit (IMU) yaw rates, and steering wheel angles. It provides the interface VehicleState to ADASController.

4.0 Communication Interfaces & Port Catalog

All inter-component communication is mediated by AUTOSAR Software Component Ports. Below is the formal interface specification matrix.

Interface Name	Provider	Consumer	Protocol	Signals
CameraData	CameraManager	SensorFusion	Ethernet	ImageFrame, TimeStamp
RadarData	RadarProcessor	SensorFusion	CAN-FD	RadarTargetList, ClusterRange
FusedObjectData	SensorFusion	ADASController	Autosar RTE	ObstacleDistance, TTC
VehicleState	VehicleInterface	ADASController	CAN	WheelSpeed, YawRate
BrakeCommand	ADASController	BrakeController	CAN	DecelerationRequest, BrakeActive

Interface Protocol Mapping Details:

• Ethernet 100BASE-T1: High bandwidth video transport between CameraManager and SensorFusion.
• CAN-FD: High-integrity radar point clusters between RadarProcessor and SensorFusion.
• Autosar RTE: Inter-core shared memory communication within ADAS Domain Controller.
• Classic CAN 2.0B: Deterministic 500kbps communication to chassis ECUs for BrakeCommand and VehicleState.

5.0 Functional Flow: Autonomous Emergency Braking (AEB)

The AEB feature execution sequence is defined as follows:

Step 1: Front camera detects obstacle and produces CameraData.
Step 2: Radar transceiver detects target range and produces RadarData.
Step 3: SensorFusion combines signals and transmits FusedObjectData to ADASController.
Step 4: ADASController evaluates vehicle speed from VehicleState and calculates TTC < 1.2s.
Step 5: ADASController issues emergency BrakeCommand.
Step 6: BrakeController actuates hydraulic calipers, achieving vehicle standstill.

Latency & Timing Budget:

Total cycle budget is 50ms. Perception pipeline contributes 25ms, Decision logic contributes 10ms, and Actuator ramp-up contributes 15ms.

6.0 Functional Safety & Requirements Traceability

In accordance with ISO 26262, the emergency braking function is allocated ASIL-D safety integrity. Software components must adhere to freedom from interference (FFI) memory partitioning.

Req ID	Component	ASIL	Safety Requirement Statement
REQ-ADAS-001	ADASController	ASIL-D	The ADASController shall command maximum emergency deceleration when TTC drops below 0.8s.
REQ-ADAS-002	SensorFusion	ASIL-B	SensorFusion shall provide fused object tracks with 99.5% confidence within 35ms of sensor acquisition.
REQ-ADAS-003	BrakeController	ASIL-D	BrakeController shall achieve 8.5 m/s^2 deceleration within 180ms of receiving BrakeCommand.

7.0 Architectural Dependency & Coupling Analysis

The software architecture enforces hierarchical, unidirectional dependencies to prevent circular deadlocks and ensure deterministic schedulability.

Documented Relationships & Communication Flows:

1. CameraManager -> SensorFusion via Ethernet
2. RadarProcessor -> SensorFusion via CAN-FD
3. SensorFusion -> ADASController via Autosar RTE
4. ADASController -> BrakeController via CAN
5. ADASController -> VehicleInterface via CAN
6. ADASController transmits data to BrakeController
7. VehicleInterface transmits data to ADASController

Component ADASController is identified as the central integration hub. Any changes to interfaces FusedObjectData, VehicleState, or BrakeCommand directly affect system schedulability.

8.0 Diagnostic & Audit Considerations

Diagnostic services follow Unified Diagnostic Services (UDS) ISO 14229 over CAN. Component health monitors report internal plausibility errors to the Dem (Diagnostic Event Manager) module.

Engineers reviewing this document must verify signal data types in AUTOSAR ARXML files against this High-Level Design baseline. All inconsistencies require human engineering evaluation.